

Université / École

Filière BDIA1 — Big Data & Intelligence Artificielle



Système Intelligent de Contrôle Qualité des Impellers

*Détection automatique de défauts de fonderie
par Deep Learning et Transfer Learning*

Réalisé par :

Ayoub EL-MOUJAHID

Mohamed ERRAHMOUNI

Soulama Doubié Judicael Noé

Hazem DOUZI

Encadré par :

Prof. Dr Naoual ATTAOUI

Année universitaire **2025–2026**

Résumé

Ce projet présente le développement d'un **système intelligent de contrôle qualité** dédié aux *impellers* (roues de pompes) issus du procédé de fonderie en moule carapace (*shell mould casting*). Après refroidissement, certaines pièces présentent des défauts (fissures, déformations, bavures) qui compromettent la fiabilité des pompes. Notre objectif est d'automatiser la détection de ces défauts par apprentissage profond.

Nous avons construit un pipeline complet de **prétraitement reproductible** (TensorFlow et PyTorch), équilibré les classes par *class weighting*, puis comparé trois familles de modèles : un **MLP** de référence, un **CNN** entraîné de zéro, et un modèle de **Transfer Learning** basé sur EFFICIENTNETB0. Le modèle final atteint **99,44 % d'accuracy** et un **AUC de 0,9993** sur l'ensemble de test, avec un *recall* de **99,34 %** sur la classe défectueuse — performance compatible avec un déploiement industriel.

Mots-clés : Contrôle qualité, fonderie, impeller, deep learning, CNN, transfer learning, EfficientNet, TensorFlow, PyTorch.

Table des matières

Résumé	1
1 Introduction et contexte industriel	4
1.1 Le procédé de fabrication des impellers	4
1.2 Problématique	4
1.3 Objectif du projet	5
2 Solution proposée	6
2.1 Pipeline de développement du modèle d'IA	6
3 Collecte et organisation des données	8
3.1 Source du dataset	8
3.2 Stratégie de répartition	8
4 Prétraitement des images	9
4.1 Vue d'ensemble du pipeline TensorFlow (10 sections)	9
4.1.1 Reproductibilité et détection du <i>device</i>	10
4.1.2 Color balancing — compatibilité <i>graph</i> TensorFlow	10
4.1.3 Normalisation <i>ImageNet</i> et pipelines train/eval	11
4.1.4 Chargement, pondération de classes et pipeline final	11
4.1.5 Configuration JSON, utilitaires et contrôles	12
4.2 Pipeline équivalent en PyTorch	13
4.2.1 Reproductibilité et device	13
4.2.2 Color balancing personnalisé	13
4.2.3 Pipeline complet d'augmentations	13
4.2.4 Gestion du déséquilibre de classes	14
4.2.5 DataLoaders et utilitaires	14
5 Modélisation et expérimentations	15
5.1 Modèle 1 — MLP (référence)	15
5.1.1 Représentation des données	15
5.1.2 Architecture du MLP	15
5.1.3 Régularisation et stratégie d'entraînement	16
5.1.4 Résultats	17
5.1.5 Analyse	17
5.2 Modèle 2 — CNN entraîné de zéro	17

5.2.1	Construction de l'architecture	17
5.2.2	Compilation et régularisation	18
5.2.3	Entraînement	19
5.2.4	Évaluation et diagnostic d'overfitting	19
5.2.5	Régularisation par augmentation des données	20
5.2.6	Conclusion intermédiaire	20
5.3	Modèle 3 — Transfer Learning avec EfficientNetB0	20
5.3.1	Qu'est-ce que le Transfer Learning ?	21
5.3.2	Stratégie en deux phases	21
5.3.3	Architecture choisie : EfficientNetB0	22
5.3.4	Source des données et préparation	22
5.3.5	Hyperparamètres	24
5.3.6	Construction du modèle	24
5.3.7	Phase 1 — Extraction de caractéristiques	24
5.3.8	Phase 2 — Fine-tuning	25
5.3.9	Courbes d'entraînement	26
6	Évaluation finale et discussion	27
6.1	Performance sur l'ensemble de test	27
6.2	Matrice de confusion	28
6.3	Courbes ROC et Precision-Recall	28
6.4	Analyse des erreurs	29
6.5	Comparaison globale des trois modèles	29
7	Conclusion et perspectives	31
7.1	Bilan	31
7.2	Perspectives	31

1. Introduction et contexte industriel

1.1 Le procédé de fabrication des impellers

Un *impeller* (ou *turbine/roue de pompe*) est une pièce métallique critique qui transmet l'énergie mécanique au fluide pompé. Sa fabrication suit trois grandes étapes :

1. **Conception (Design of pump).** La pièce finale est modélisée en CAO : une roue à aubes destinée à équiper une pompe centrifuge.
2. **Moule carapace (Shell mould).** Un moule en sable aggloméré par résine est confectionné autour de l'empreinte de la pièce.
3. **Coulée en moule carapace (Shell mould casting).** Plusieurs moules sont assemblés et serrés dans un cadre. Le métal en fusion est coulé, refroidit et se solidifie. Après démoulage, la pièce métallique est nettoyée, ébavurée puis usinée si nécessaire.



(a) Moule carapace assemblé

(b) Coulée et démoulage

Figure 1.1 – Étapes du procédé de fonderie en moule carapace.

1.2 Problématique

Après refroidissement, le contrôle visuel manuel de chaque pièce est :

- lent et coûteux (cadence de production limitée),
- subjectif (dépendant de l'opérateur),
- sujet à erreurs (fatigue, défauts subtils non vus à l'œil nu).

Les anomalies les plus fréquentes sont : **fissures**, **déformations** et **bavures**. Un seul défaut non détecté peut provoquer une défaillance de pompe en service, avec des conséquences coûteuses pour le client final.

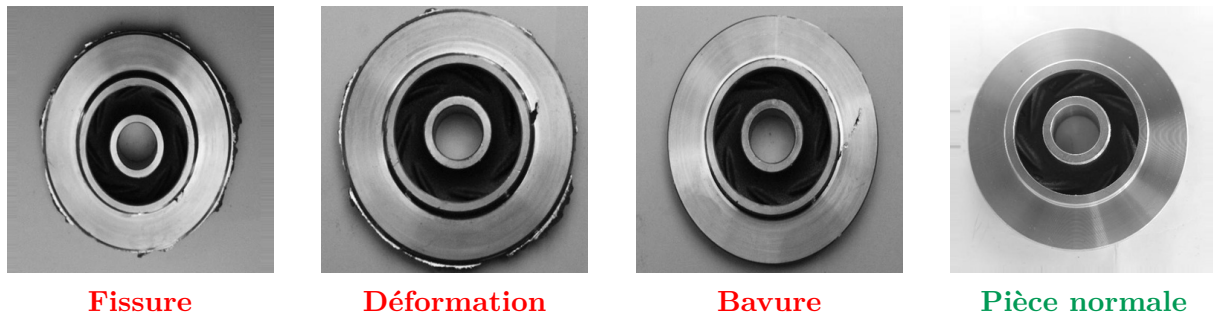


Figure 1.2 – Exemples typiques de défauts *vs.* pièce conforme.

1.3 Objectif du projet

Objectif

Mettre en place un **système de contrôle qualité automatique** fondé sur la vision par ordinateur et le *deep learning*, capable de classifier en temps réel chaque impeller comme `def_front` (défectueux) ou `ok_front` (conforme), avec un **recall élevé sur la classe défectueuse**.

2. Solution proposée

Notre solution repose sur une chaîne de traitement entièrement logicielle qui s'intègre en aval de la ligne de fonderie. Une caméra acquiert l'image de chaque pièce, qui est ensuite analysée par un modèle d'IA renvoyant la prédiction *conforme/défectueux* en quelques millisecondes.



Figure 2.1 – Architecture globale du système de contrôle qualité proposé.

2.1 Pipeline de développement du modèle d'IA

Le développement du modèle suit cinq étapes successives illustrées en figure 2.2 :

1. Collecte des images d'impellers,
2. Prétraitement des images,
3. Entraînement du modèle,
4. Détection et classification des défauts,
5. Test et validation.

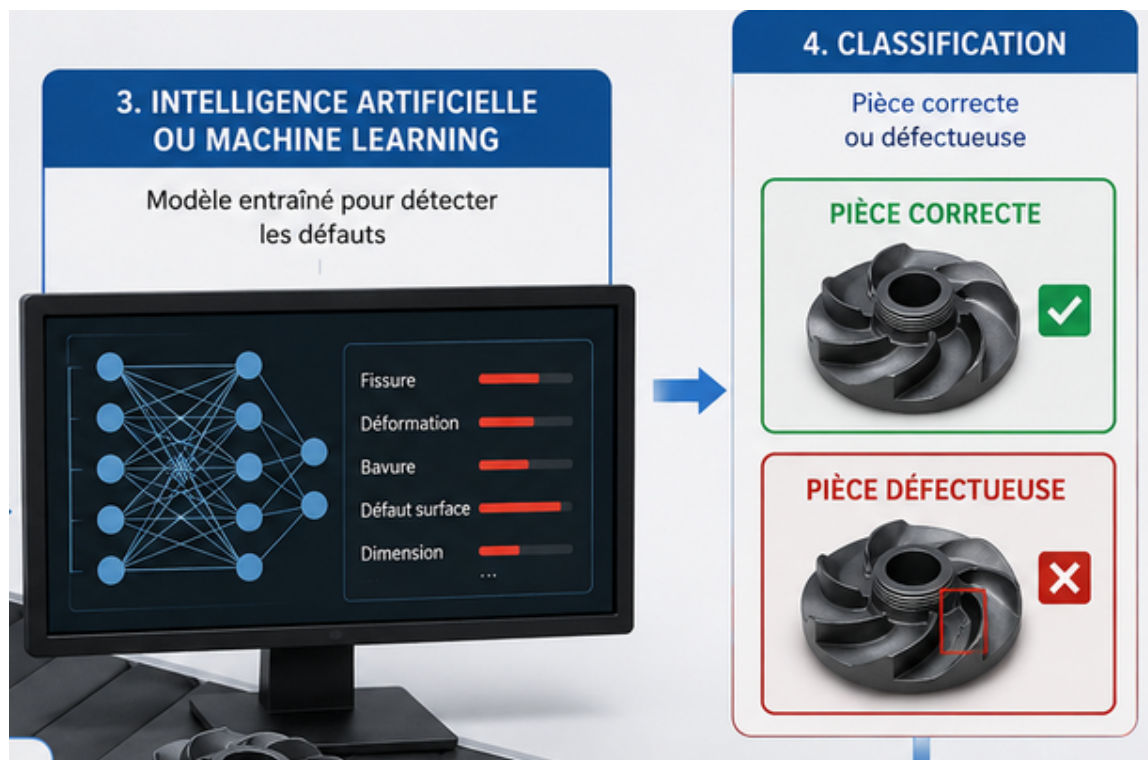


Figure 2.2 – Pipeline de développement du modèle d'intelligence artificielle.

3. Collecte et organisation des données

3.1 Source du dataset

Le jeu de données utilisé est le *Casting Defect Detection* créé par Ravirajsinh Dabhi et publié sur Kaggle. Il contient des images de pièces de fonderie réparties en deux classes :

- `def_front` — pièces présentant un défaut visible,
- `ok_front` — pièces conformes.

Le dataset est déjà organisé en trois sous-ensembles *train* / *validation* / *test*, ce qui facilite la mise en place du protocole expérimental.

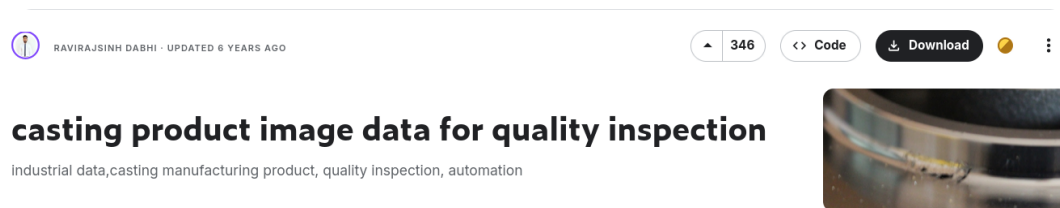


Figure 3.1 — Aperçu du dataset *Casting Defect Detection* (Kaggle).

3.2 Stratégie de répartition

Ensemble	Proportion	Rôle
Train	80 %	Apprentissage des paramètres
Validation	10 %	Réglage des hyperparamètres / <i>early stopping</i>
Test	10 %	Évaluation finale (jamais vue)

Table 3.1 — Répartition train/validation/test du dataset.

4. Prétraitement des images

Nous avons développé deux pipelines équivalents : un en **TensorFlow** (`tf.data`) et un en **PyTorch** (`DataLoader`). Tous deux partagent la même philosophie : reproductibilité, normalisation *ImageNet*, augmentation stochastique sur le *train*, et prétraitement déterministe sur *validation/test*.

4.1 Vue d'ensemble du pipeline TensorFlow (10 sections)

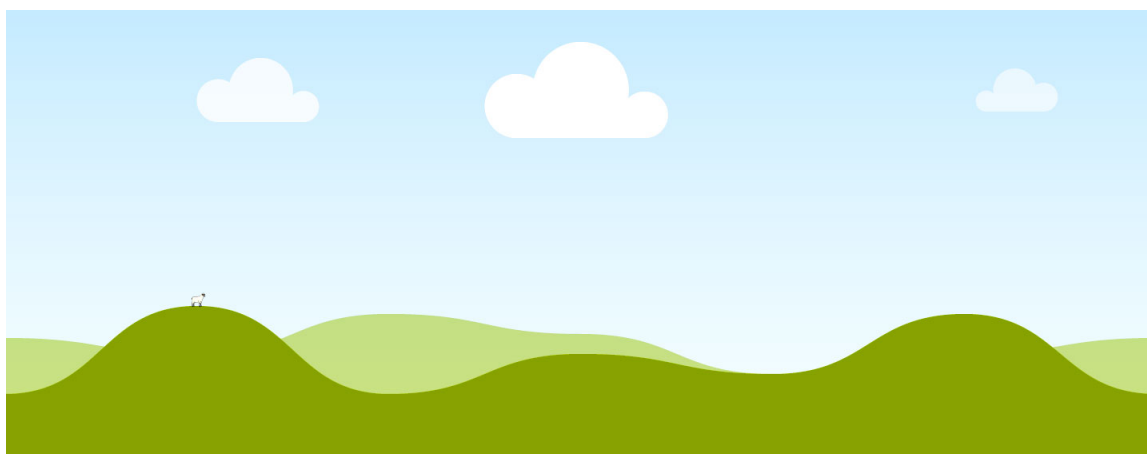


Figure 4.1 – Bandeau du pipeline de prétraitement TensorFlow.

1. **Reproductibilité** : `SEED = 42 (random, numpy, tensorflow)`.
2. **Détection GPU/CPU** : `tf.config.list_physical_devices()`.
3. **Color Balancing** : White Balance, Histogram Equalization, Channel Shuffle.
4. **Normalisation ImageNet** : $\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$.
5. **Pipeline Train** : crop 224×224 , flips, rotations, jitter.
6. **Pipeline Eval** : *center crop* déterministe + white balance.
7. **Chargement** : `image_dataset_from_directory` pour train/val/test.
8. **Class Weights** : équivalent du `WeightedRandomSampler` de PyTorch.
9. **Pipelines tf.data** : `.map()` $\rightarrow$ `.batch()` $\rightarrow$ `.prefetch(AUTOTUNE)`.

10. **Configuration** : export JSON, sanity check, visualisation.

4.1.1 Reproductibilité et détection du *device*

```

1 SEED = 42
2 random.seed(SEED)
3 np.random.seed(SEED)
4 tf.random.set_seed(SEED)
5
6 gpus = tf.config.list_physical_devices("GPU")
7 DEVICE = "GPU" if gpus else "CPU"
8 if gpus:
9     for gpu in gpus:
10         tf.config.experimental.set_memory_growth(gpu, True)

```

Listing 4.1 – Graine globale et détection GPU/CPU.

Constante	Valeur	Rôle
IMG_SIZE	224	Côté image (carrée)
BATCH_SIZE	32	Images par batch
NUM_CLASSES	2	<i>defect</i> / <i>ok_front</i>
MEAN	[0.485, 0.456, 0.406]	Moyenne ImageNet (R/G/B)
STD	[0.229, 0.224, 0.225]	Écart-type ImageNet (R/G/B)
AUTOTUNE	tf.data.AUTOTUNE	Parallélisme automatique

Table 4.1 – Constantes globales partagées par tous les notebooks.

Hyperparamètres globaux.

4.1.2 Color balancing — compatibilité *graph* TensorFlow

Trois transformations stochastiques compatibles avec le *graph mode* :

Gray-World White Balance ($p = 0,5$) normalise chaque canal par sa moyenne ; le facteur d'échelle est borné dans $[0,5 ; 2,0]$ pour éviter la sur-exposition. Implémentée nativement avec `tf.cond`.

Histogram Equalization ($p = 0,3$) égalise l'histogramme canal par canal via `PIL.ImageOps.equalize` encapsulée dans `tf.py_function`.

Channel Shuffle ($p = 0,15$) permute aléatoirement l'ordre des canaux RGB (6 permutations), pour rendre le modèle robuste aux dérives de balance des couleurs.

```

1 means = tf.reduce_mean(image, axis=[0, 1])
2 global_mean = tf.reduce_mean(means)
3 scale = global_mean / means # (3,)
4 scale = tf.clip_by_value(scale, 0.5, 2.0)
5 return tf.clip_by_value(image * scale, 0., 1.)

```

Listing 4.2 – Gray-World White Balance.

4.1.3 Normalisation *ImageNet* et pipelines train/eval

$$\hat{x} = \frac{x - \mu}{\sigma}, \quad \mu = \begin{pmatrix} 0.485 \\ 0.456 \\ 0.406 \end{pmatrix}, \quad \sigma = \begin{pmatrix} 0.229 \\ 0.224 \\ 0.225 \end{pmatrix} \quad (4.1)$$

Opération	preprocess_train	preprocess_eval
Resize	256 × 256	256 × 256
Crop	random 224 × 224	center crop 224 × 224
Flips	H + V	—
Rotation	±180° (tfa)	—
Color balance	stochastique	déterministe ($p = 1$)
Color jitter	brightness/contrast/sat/hue	—
Grayscale	$p = 0,1$	—
Normalisation	ImageNet	ImageNet

Table 4.2 – Comparaison *train vs. eval*.

4.1.4 Chargement, pondération de classes et pipeline final

```

1 def _load_raw(directory, shuffle):
2     return tf.keras.utils.image_dataset_from_directory(
3         directory, labels="inferred", label_mode="int",
4         color_mode="rgb", batch_size=None,
5         image_size=(256, 256), shuffle=shuffle, seed=SEED,
6     )
7
8 raw_train = _load_raw(TRAIN_DIR, shuffle=True)
9 raw_val   = _load_raw(VAL_DIR,  shuffle=False)
10 raw_test  = _load_raw(TEST_DIR, shuffle=False)
11
12 all_labels = np.array([lbl.numpy() for _, lbl in raw_train])
13 class_counts = np.bincount(all_labels, minlength=NUM_CLASSES)
14 total       = len(all_labels)

```

```

15 class_weights = {
16     i: float(total) / (NUM_CLASSES * int(count))
17     for i, count in enumerate(class_counts)
18 }

```

Listing 4.3 – Chargement `tf.data` et pondération des classes.

$$w_i = \frac{N_{\text{total}}}{N_{\text{classes}} \cdot n_i} \quad (4.2)$$

```

1 train_dataset = (raw_train
2     .map(preprocess_train, num_parallel_calls=AUTOTUNE)
3     .batch(BATCH_SIZE, drop_remainder=True)
4     .prefetch(AUTOTUNE))
5
6 val_dataset = (raw_val
7     .map(preprocess_eval, num_parallel_calls=AUTOTUNE)
8     .batch(BATCH_SIZE).prefetch(AUTOTUNE))
9
10 test_dataset = (raw_test
11     .map(preprocess_eval, num_parallel_calls=AUTOTUNE)
12     .batch(BATCH_SIZE).prefetch(AUTOTUNE))

```

Listing 4.4 – Composition finale des datasets.

4.1.5 Configuration JSON, utilitaires et contrôles

```

1 CONFIG = {
2     "img_size"      : 224,
3     "batch_size"    : 32,
4     "num_classes"   : 2,
5     "class_names"   : ["defect", "ok_front"],
6     "mean"          : [0.485, 0.456, 0.406],
7     "std"           : [0.229, 0.224, 0.225],
8     "class_weights" : {"0": 1.12, "1": 0.90},
9     "device"        : "GPU"
10 }

```

Listing 4.5 – Export centralisé de la configuration.

Quatre utilitaires complètent le module `preprocessing_tf` :

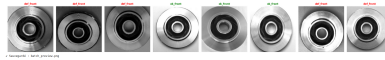
- `sanity_check()` — affiche *shape*, min/max/mean d'un batch.
- `show_batch(ds, n=8)` — visualise *n* images avec leur label.
- `plot_class_distribution()` — histogramme *defect vs. ok_front*.

— `compute_mean_std(ds, n=20)` — moyenne et écart-type empiriques.

```

— Sanity check —
Batch shape : (32, 224, 224, 3)
Label shape : [0, 0, 1, 1, 0, 0, 1, 0]
Pixel min   : -2.118
Pixel max   : 2.649
Pixel mean  : 0.299

```



(a) `sanity_check()`

(b) `show_batch()`

(c) Distribution des classes

Figure 4.2 — Sorties des utilitaires de contrôle.

4.2 Pipeline équivalent en PyTorch

Le pipeline PyTorch reprend les mêmes étapes (reproductibilité, augmentations, normalisation ImageNet) en s'appuyant sur `torchvision.transforms` et un `DataLoader` avec `WeightedRandomSampler`.

4.2.1 Reproductibilité et device

```

1 SEED = 42
2 random.seed(SEED); np.random.seed(SEED)
3 torch.manual_seed(SEED); torch.cuda.manual_seed_all(SEED)
4 torch.backends.cudnn.deterministic = True
5 torch.backends.cudnn.benchmark = False
6
7 DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

```

Listing 4.6 — Reproductibilité et détection automatique du device.

4.2.2 Color balancing personnalisé

- **AutoWhiteBalance** ($p = 0,5$) — hypothèse *Gray-World* pour corriger les dominantes des caméras industrielles.
- **HistogramEqualization** ($p = 0,3$) — fait ressortir les défauts subtils dans les zones peu contrastées.
- **ChannelShuffle** ($p = 0,15$) — permute les canaux RGB pour rendre le modèle indépendant de la teinte.

4.2.3 Pipeline complet d'augmentations

Les transformations appliquées sur le *train* se composent comme suit : `Resize(256)`, `RandomCrop(224)`, `RandomHorizontalFlip`, `RandomVerticalFlip`, `RandomRotation(180)`,

AutoWhiteBalance, HistogramEqualization, ChannelShuffle, ColorJitter, RandomAffine(translation, scale), RandomGrayscale(0.1), GaussianBlur, ToTensor, et enfin Normalize(μ, σ).

4.2.4 Gestion du déséquilibre de classes

Le dataset présente un léger déséquilibre (environ 65 % de `def_front` et 35 % de `ok_front`). Sans correction, le modèle apprendrait à toujours prédire la classe majoritaire. Nous utilisons un `WeightedRandomSampler` :

```

1 class_counts    = torch.bincount(targets)
2 class_weights   = 1.0 / class_counts
3 sample_weights  = class_weights[targets]
4 sampler = WeightedRandomSampler(
5     sample_weights, num_samples=len(sample_weights),
6     replacement=True
7 )

```

Listing 4.7 – Pondération inverse-fréquence des échantillons.

4.2.5 DataLoaders et utilitaires

Loader	batch_size	shuffle	Sampler	pin_memory
train_loader	32	—	WeightedRandom	si CUDA
val_loader	32	False	—	si CUDA
test_loader	32	False	—	si CUDA

Table 4.3 – Configuration des trois DataLoaders.

5. Modélisation et expérimentations

Trois familles de modèles ont été comparées dans une démarche progressive :

1. Un **MLP** (Multi-Layer Perceptron) comme *baseline*.
2. Un **CNN** entraîné de zéro pour exploiter la structure spatiale des images.
3. Un modèle de **Transfer Learning** fondé sur EFFICIENTNETB0.

5.1 Modèle 1 — MLP (référence)

5.1.1 Représentation des données

Les images d'entrée sont au format $224 \times 224 \times 3$. Or, un MLP ne sait traiter que des vecteurs 1D. Une couche **Flatten** est donc nécessaire :

$$(224 \times 224 \times 3) \longrightarrow \mathbb{R}^{150\,528} \quad (5.1)$$

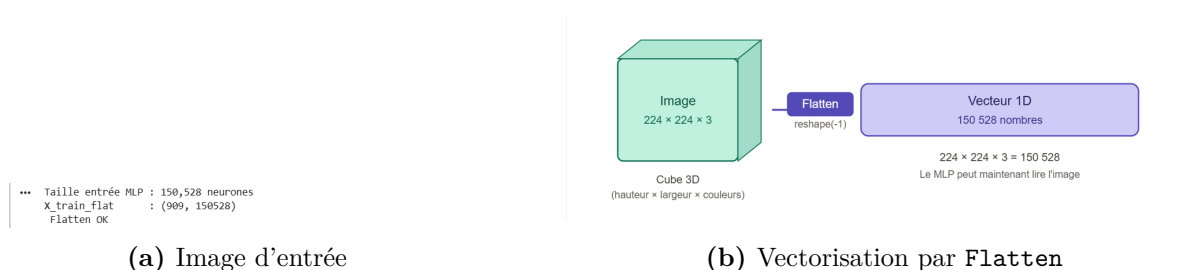


Figure 5.1 — Aplatissement d'une image en vecteur 1D.

5.1.2 Architecture du MLP

Architecture du MLP

Entrée : 150 528 neurones

- Dense(512) + BatchNorm + ReLU + Dropout(0.4)
- Dense(256) + BatchNorm + ReLU + Dropout(0.4)
- Dense(128) + BatchNorm + ReLU + Dropout(0.3)
- Dense(2) + **Softmax** (→ `def_front` / `ok_front`)

Model: "MLP"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 512)	77,070,848
batch_normalization	(None, 512)	2,048
activation (ReLU)	(None, 512)	0
dropout (Dropout 0.4)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131,328
batch_normalization_1	(None, 256)	1,024
activation_1 + dropout_1 (0.4)	(None, 256)	0
dense_2 (Dense)	(None, 128)	32,896
batch_normalization_2	(None, 128)	512
activation_2 + dropout_2 (0.3)	(None, 128)	0
dense_3 (Dense + Softmax)	(None, 2)	258

Total params: 77,238,914 (294.64 MB)
Trainable params: 77,237,122 (294.64 MB)
Non-trainable: 1,792 (7.00 KB)

Figure 5.2 – Schéma global de l'architecture MLP.

5.1.3 Régularisation et stratégie d'entraînement

- **Batch Normalization** : recentre les activations entre couches, accélère et stabilise l'apprentissage.
- **Dropout (0,3–0,4)** : éteint aléatoirement des neurones pour éviter le sur-apprentissage.
- **Early Stopping (patience = 7)** : arrêt automatique en cas de stagnation sur la validation.
- **ReduceLROnPlateau** : division du *learning rate* si la perte stagne.

5.1.4 Résultats

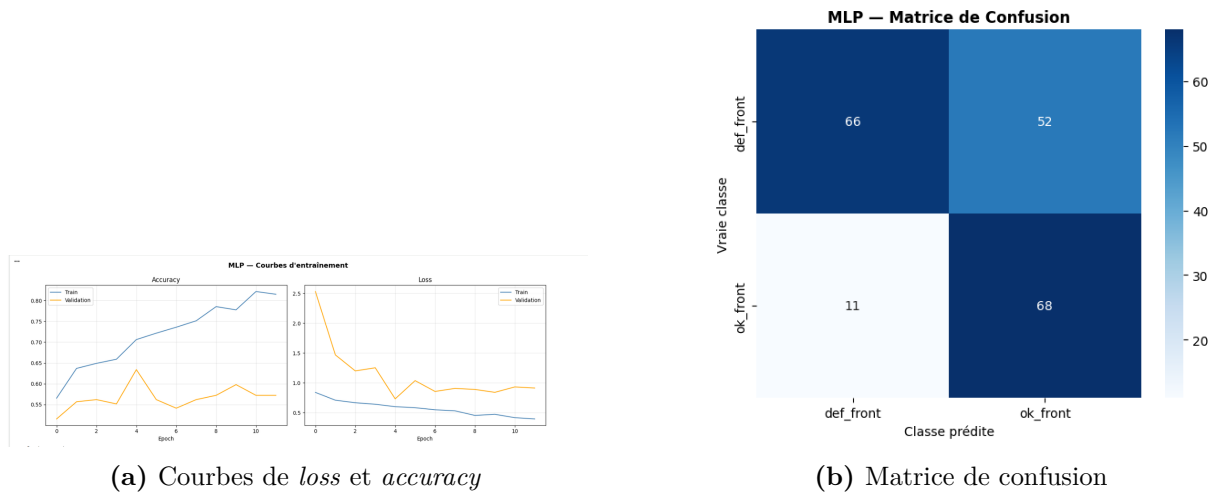


Figure 5.3 – Performance du MLP sur le jeu de test.

Métrique	Valeur
Test Loss	0,7587
Test Accuracy	68,02 %
Precision	74,07 %
Recall	68,02 %
F1-Score	67,95 %
Epochs entraînés	12/30 (early stopping déclenché)

Table 5.1 – Résultats du MLP sur l'ensemble de test.

5.1.5 Analyse

Points forts. Convergence rapide, modèle de référence simple à comprendre et à entraîner.

Limites. Le Flatten détruit la structure spatiale : les pixels voisins perdent leur lien, le modèle ne peut pas apprendre de *textures* ni de *contours*. D'où la nécessité de passer à un CNN.

5.2 Modèle 2 — CNN entraîné de zéro

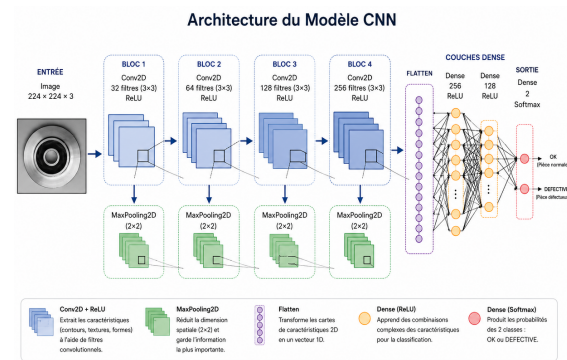
5.2.1 Construction de l'architecture

Le CNN exploite la structure 2D des images via des couches de convolution suivies de *pooling*, qui extraient progressivement des caractéristiques de complexité croissante (bords

→ textures → formes).

```
model_cnn = models.Sequential([
    layers.Input(shape=(224, 224, 3)),
    # Block 1
    layers.Conv2D(32, (3,3), activation='relu',padding='same'),
    layers.MaxPooling2D((2,2)),
    #Block 2
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    #Block 3
    layers.Conv2D(128, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    #Block 4
    layers.Conv2D(256, (3,3), activation='relu'),
    layers.MaxPooling2D((2,2)),
    #Flatten
    layers.Flatten(),
    # Dense layers
    layers.Dense(256, activation='relu'),
    layers.Dense(128, activation='relu'),
    # Output
    layers.Dense(2, activation='softmax')])
```

(a) Bloc convolutif générique



(b) Vue d'ensemble du CNN

Figure 5.4 – Architecture du CNN développé.

5.2.2 Compilation et régularisation

- **Early Stopping** pour éviter le sur-apprentissage.
- **Batch Normalization** pour stabiliser et accélérer l'apprentissage.
- **Dropout** pour réduire le sur-apprentissage en désactivant aléatoirement des neurones.

```
model_cnn.compile(optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"])
```

Figure 5.5 – Compilation du modèle CNN (*loss categorical crossentropy*, optimiseur Adam).

5.2.3 Entraînement

```
model_cnn_reg = models.Sequential([
    #input
    layers.Input(shape=(224, 224, 3)),
    #Block 1
    layers.Conv2D(32, (3,3), padding='same'),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2,2)),
    #Block 2
    layers.Conv2D(64, (3,3), padding='same'),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2,2)),
    #Block 3
    layers.Conv2D(128, (3,3), padding='same'),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2,2)),
    #Block 4
    layers.Conv2D(256, (3,3), padding='same'),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.MaxPooling2D((2,2)),
    #flatten
    layers.Flatten(),
    #Dense layers
    layers.Dense(256),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Dropout(0.5),
    layers.Dense(128),
    layers.BatchNormalization(),
    layers.Activation('relu'),
    layers.Dropout(0.3),
    # Output
    layers.Dense(2, activation='softmax')])
```

(a) Boucle d'entraînement

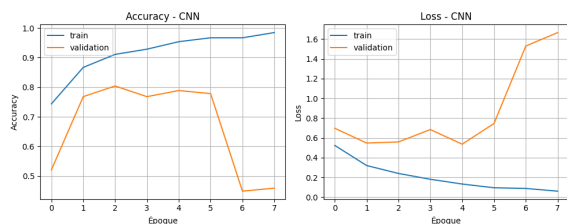
```
early_stop_cnn = callbacks.EarlyStopping(
    monitor="val_loss",
    patience=3,
    restore_best_weights=True
)

history_cnn_reg = model_cnn_reg.fit(
    train_loader,
    validation_data=val_loader,
    epochs=20,
    callbacks=[early_stop_cnn],
    verbose=1
)
```

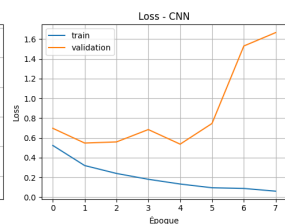
(b) Logs Keras (extrait)

Figure 5.6 – Entraînement du CNN.

5.2.4 Évaluation et diagnostic d'overfitting



(a) Accuracy train vs validation



(b) Loss train vs validation

```
test_loss_cnn_reg, test_acc_cnn_reg = model_cnn_reg.evaluate(
    test_loader,
    verbose=0
)

print("Test Loss:", test_loss_cnn_reg)
print("Test Accuracy:", test_acc_cnn_reg)

Test Loss: 0.6228603720664978
Test Accuracy: 0.8020304441452026
```

Figure 5.7 – Courbes d'entraînement du CNN avant régularisation supplémentaire.

Interprétation.

- L'*accuracy* de **train** monte jusqu'à $\sim 99\%$: le modèle apprend très bien les données d'entraînement.
- L'*accuracy* de **validation** monte d'abord, puis **diminue** en fin d'entraînement.
- La *loss* de validation **remonte** à partir des dernières *epochs*.

⇒ Diagnostic clair de **sur-apprentissage** (*overfitting*) : le modèle mémorise les données d'entraînement.

5.2.5 Régularisation par augmentation des données

Pour atténuer l'overfitting, nous ajoutons des augmentations de données (rotations, flips, jitter, zoom).

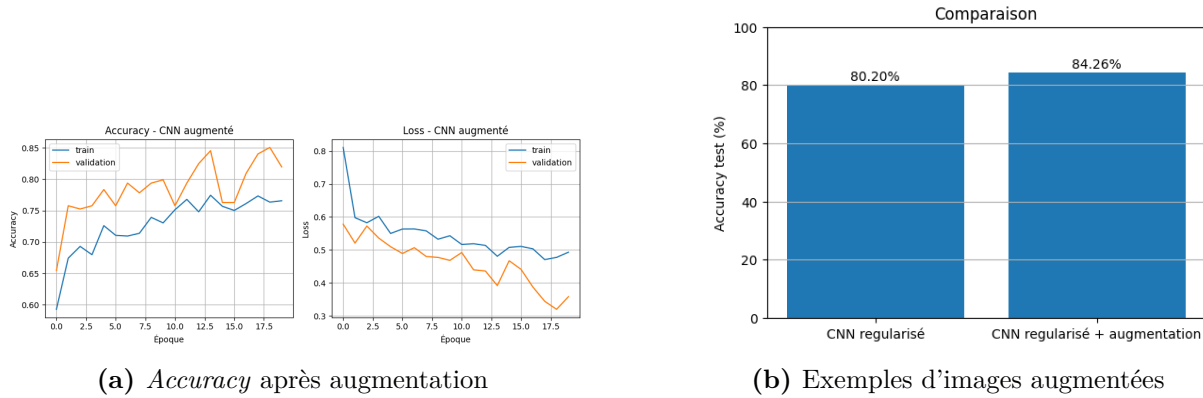


Figure 5.8 – Effet de la *data augmentation* sur l'apprentissage.

5.2.6 Conclusion intermédiaire

Bilan du CNN

L'augmentation des données corrige efficacement le problème d'*overfitting* et fournit un modèle plus robuste et plus généralisable. Cependant, atteindre 99% de précision avec un CNN entraîné de zéro nécessiterait beaucoup plus de données. C'est pourquoi nous passons au **Transfer Learning**.

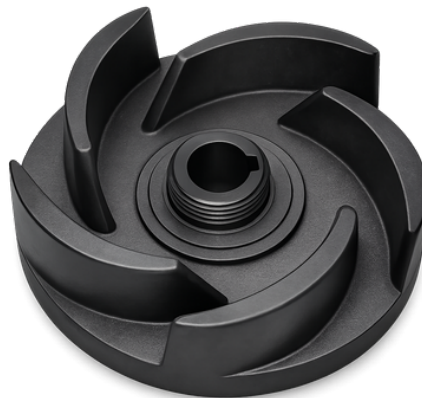


Figure 5.9 – Synthèse du modèle CNN.

5.3 Modèle 3 — Transfer Learning avec EfficientNetB0

5.3.1 Qu'est-ce que le Transfer Learning ?

Le *Transfer Learning* (apprentissage par transfert) consiste à **réutiliser un réseau de neurones déjà entraîné** sur une grande base d'images (ImageNet, $\sim 1,2$ million d'images, 1 000 classes), puis à l'adapter à un nouveau problème plus spécifique — ici, la détection de défauts de fonderie.

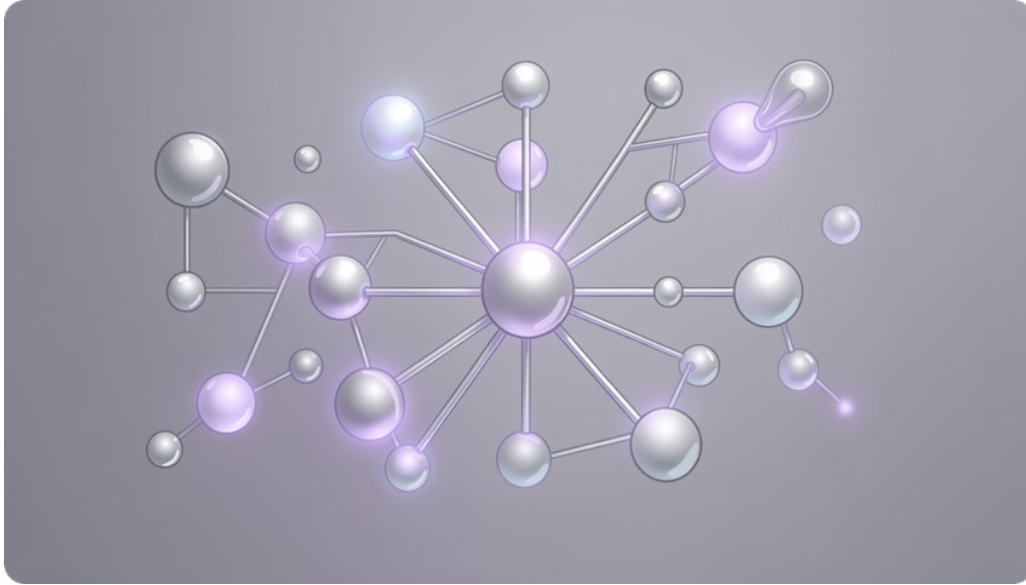


Figure 5.10 – Principe du Transfer Learning.

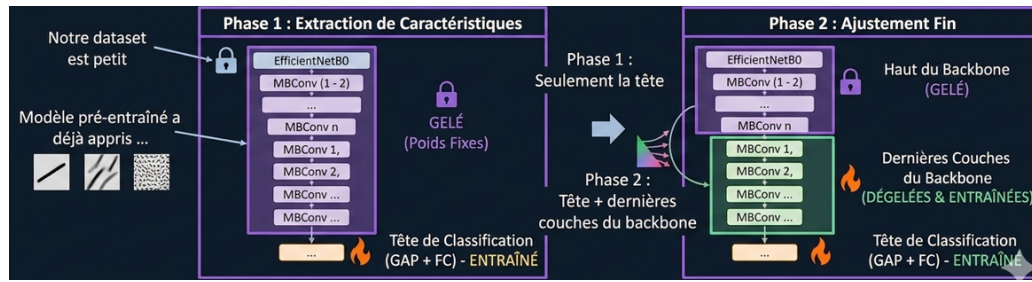
5.3.2 Stratégie en deux phases

Phase	Ce que l'on entraîne
1. Extraction de caractéristiques	Tête de classification uniquement : le <i>backbone</i> pré-entraîné est gelé.
2. <i>Fine-tuning</i>	Tête + dernières couches du <i>backbone</i> , avec un <i>learning rate</i> très faible.

Table 5.2 – Stratégie de Transfer Learning à deux étages.

Pourquoi ce choix ?

- Notre dataset est relativement petit ($\sim 6\,600$ images d'entraînement).
- Entraîner un grand réseau de zéro provoquerait du sur-apprentissage.
- Les caractéristiques visuelles génériques apprises sur ImageNet (contours, textures, formes) sont *transférables* à presque toute tâche d'image.

Figure 5.11 – Réutilisation du *backbone* pré-entraîné.

5.3.3 Architecture choisie : EfficientNetB0

EFFICIENTNET est une famille de modèles de *deep learning* pour la classification d'images, développée par Google en 2019. Sa particularité est de **coupler haute performance et faible coût de calcul** grâce à un *compound scaling* (mise à l'échelle conjointe de la profondeur, largeur et résolution).

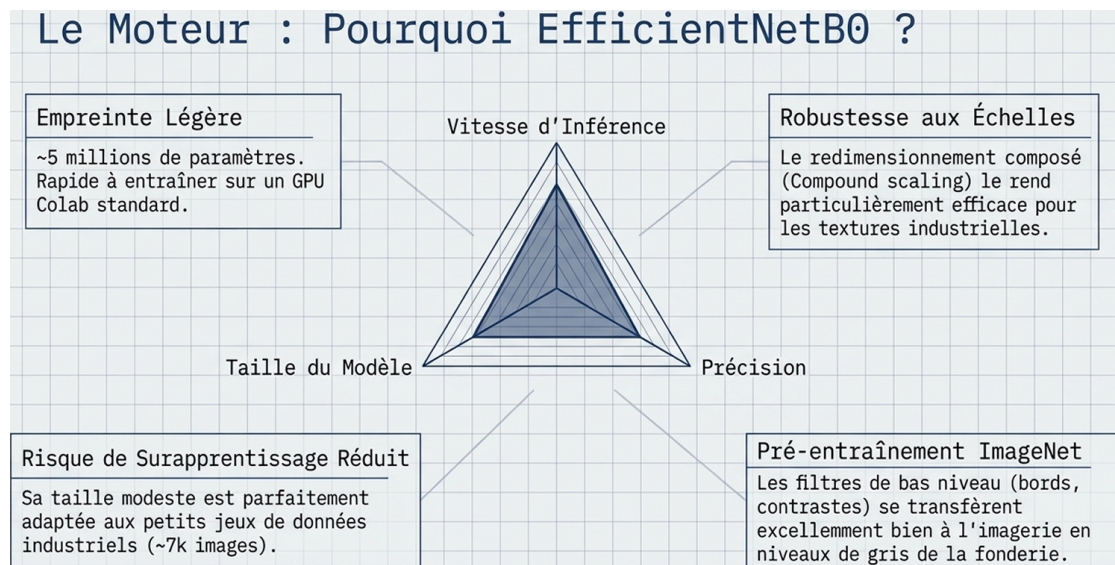


Figure 5.12 – Architecture EfficientNetB0.

5.3.4 Source des données et préparation

Le dataset utilisé est la **version augmentée** fournie directement par l'auteur sur Kaggle, déjà adaptée aux contraintes industrielles (variations d'éclairage, angles, textures de surface). Notre prétraitement se limite donc au redimensionnement et à la normalisation requise par EfficientNetB0.

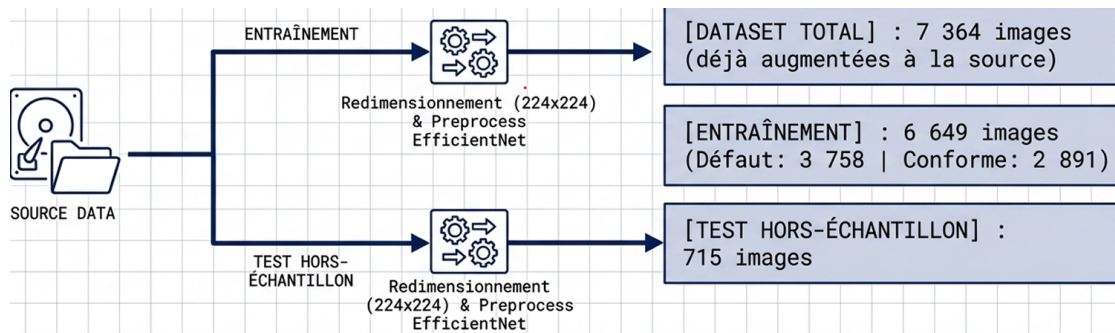


Figure 5.13 – Dataset augmenté fourni sur Kaggle.

Chargement automatique. Keras lit l'arborescence `train/ok_front/` et `train/def_front/`, redimensionne en 224×224 et attribue un label binaire automatiquement. La fonction `preprocess_input` d'EfficientNet transforme les valeurs $[0, 255]$ vers $[-1, 1]$.

Préparation des données

a) Chargement depuis les dossiers

```

train_ds_raw = keras.utils.image_dataset_from_directory(
    TRAIN_DIR,
    labels='inferred',          # Les labels sont déduits du nom des dossiers
    label_mode='binary',       # 0 = bonne pièce, 1 = défectueuse
    image_size=(224, 224),     # redimensionne-la en un carré de 224 pixels de côté
    batch_size=32,
    shuffle=True,              # oblige à rester attentive aux détails plutôt qu'à l'ordre des images.
    validation_split=0.15,
    subset='training',
)
  
```

Keras lit automatiquement les images depuis l'arborescence `train/ok_front/` et `train/def_front/`, les redimensionne en 224×224 , et leur attribue un label binaire.

b) Prétraitement spécifique à EfficientNet

```

preprocess = keras.applications.efficientnet.preprocess_input

def _prep(x, y):
    return preprocess(x), y

train_ds = train_ds_raw.map(_prep).cache().prefetch(AUTOTUNE)
  
```

Transforme automatiquement tes photos de "0 à 255" vers le format "-1 à 1" et optimise le processus du l'entraînement

Figure 5.14 – Code de chargement et de prétraitement.

5.3.5 Hyperparamètres

```

IMG_SIZE      = 224      # taille d'entrée d'EfficientNetB0: conçus pour regarder des images à travers une fenêtre
                        # d'une taille très précise.

BATCH_SIZE    = 32      # C'est le nombre d'images que le modèle regarde en même temps avant de faire un petit
                        # bilan et de s'améliorer.

VAL_SPLIT     = 0.15     # 15 % du train pour la validation: Ici, il s'entraîne sur 85% des images et fait un "test
                        # blanc" sur les 15% restants.

EPOCHS_HEAD   = 10      # max époques en Phase 1: Nombre de passages complets sur la totalité des données
                        # d'entraînement.

EPOCHS_FT     = 10      # max époques en Phase 2: Comme "EPOCHS_HEAD", mais pour la deuxième phase d'apprentissage
                        # (fine-tuning).

LR_HEAD       = 1e-3     # taux d'apprentissage Phase 1: En gros, c'est la vitesse à laquelle le modèle corrige ses
                        # erreurs. 1e-3 (soit 0.001) est un rythme standard pour commencer.

LR_FT         = 1e-5     # taux d'apprentissage Phase 2 (100x plus petit !): On le réduit drastiquement pour ne pas
                        # "casser" ce que le modèle a appris.

N_UNFREEZE    = 40      # nombre de couches dégelées en Phase 2: On autorise seulement les 40 dernières couches à
                        # apprendre. Les premières couches (qui détectent les contours et textures de base) restent figées.

```

Figure 5.15 – Hyperparamètres clés des deux phases.

5.3.6 Construction du modèle

On conserve le *backbone* EfficientNetB0 sans son classificateur d'origine, et on le **verrouille** pour préserver le savoir acquis sur ImageNet. Les cartes de caractéristiques sont condensées par un *Global Average Pooling*, suivies d'un *Dropout* pour la régularité, et d'une couche *Dense(1)* avec activation *sigmoid* pour la prédiction binaire.

```

def build_model():
    base_model = keras.applications.EfficientNetB0(
        include_top=False, # on enlève la tête ImageNet (1000 classes)
        weights='imagenet', # on garde les poids appris sur ImageNet
        input_shape=(IMG_SIZE, IMG_SIZE, 3),
    )
    base_model.trainable = False # ON GÈLE le modèle de base pour ne perdre pas ses poids

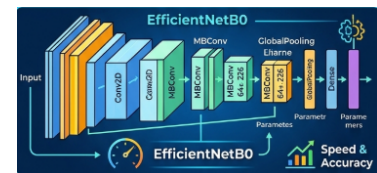
    # On construit notre propre tête de classification
    inputs = keras.Input(shape=(IMG_SIZE, IMG_SIZE, 3))
    x = base_model(inputs, training=False) # l'Image passe par le modèle de base sans changer ses poids.
    x = layers.GlobalAveragePooling2D()(x) # Elle transforme toutes ces observations complexes en une simple liste de chiffres (un vecteur).
    x = layers.Dropout(0.3)(x) # Régularisation: 30 % des neurones désactivés pour éviter la sur-apprentissage
    outputs = layers.Dense(1, activation='sigmoid')(x) # 1 sortie: proba de défaut
    model = keras.Model(inputs, outputs, name='effnetb0_casting')
    return model, base_model

model, base_model = build_model()

model.compile(
    optimizer=keras.optimizers.Adam(LR_HEAD), # Adam optimizer c'est lui qui décide comment ajuster les réglages en fonction des erreurs.
    loss='binary_crossentropy', # cette fonction calcule à quel point elle s'est trompée pour la punir et l'aider à se corriger.
    metrics=['accuracy', keras.metrics.AUC(name='auc')],
)
model.summary()

```

(a) Code de construction du modèle



(b) Sortie

Figure 5.16 – Construction du modèle de Transfer Learning.

5.3.7 Phase 1 — Extraction de caractéristiques

Seule la tête de classification est entraînée (1280 paramètres). Les résultats sont déjà excellents :

Phase 1 — Résultats

accuracy :0,9749 | auc :0,9957 | loss :0,0924
 val_accuracy :0,9900 | val_auc :0,9963 | val_loss :0,0625

```
Epoch 1/10
177/177 ----- 649s 35s/step - accuracy: 0.8422 - auc: 0.9297 - loss: 0.4151 - val_accuracy: 0.9709 - val_auc: 0.9935 - val_loss: 0.2445 - learning_rate: 0.0010
Epoch 2/10
177/177 ----- 6s 36ms/step - accuracy: 0.9397 - auc: 0.9851 - loss: 0.2372 - val_accuracy: 0.9829 - val_auc: 0.9942 - val_loss: 0.1647 - learning_rate: 0.0010
Epoch 3/10
177/177 ----- 7s 38ms/step - accuracy: 0.9551 - auc: 0.9900 - loss: 0.1831 - val_accuracy: 0.9870 - val_auc: 0.9947 - val_loss: 0.1278 - learning_rate: 0.0010
Epoch 4/10
177/177 ----- 6s 36ms/step - accuracy: 0.9616 - auc: 0.9929 - loss: 0.1515 - val_accuracy: 0.9870 - val_auc: 0.9951 - val_loss: 0.1095 - learning_rate: 0.0010
Epoch 5/10
177/177 ----- 10s 36ms/step - accuracy: 0.9678 - auc: 0.9937 - loss: 0.1354 - val_accuracy: 0.9890 - val_auc: 0.9954 - val_loss: 0.0928 - learning_rate: 0.0010
Epoch 6/10
177/177 ----- 7s 37ms/step - accuracy: 0.9683 - auc: 0.9941 - loss: 0.1205 - val_accuracy: 0.9900 - val_auc: 0.9956 - val_loss: 0.0844 - learning_rate: 0.0010
Epoch 7/10
177/177 ----- 6s 36ms/step - accuracy: 0.9722 - auc: 0.9944 - loss: 0.1131 - val_accuracy: 0.9890 - val_auc: 0.9958 - val_loss: 0.0776 - learning_rate: 0.0010
Epoch 8/10
177/177 ----- 7s 38ms/step - accuracy: 0.9720 - auc: 0.9950 - loss: 0.1057 - val_accuracy: 0.9890 - val_auc: 0.9961 - val_loss: 0.0718 - learning_rate: 0.0010
Epoch 9/10
177/177 ----- 7s 37ms/step - accuracy: 0.9738 - auc: 0.9955 - loss: 0.0987 - val_accuracy: 0.9900 - val_auc: 0.9961 - val_loss: 0.0664 - learning_rate: 0.0010
Epoch 10/10
177/177 ----- 6s 36ms/step - accuracy: 0.9749 - auc: 0.9957 - loss: 0.0924 - val_accuracy: 0.9900 - val_auc: 0.9963 - val_loss: 0.0625 - learning_rate: 0.0010
```

Figure 5.17 — Logs d'entraînement — Phase 1.

Point clé. La *val_accuracy* (99 %) est **supérieure** à l'*accuracy* de train (97,5 %) — signe que le modèle *généralise* très bien et ne sur-apprend pas. Les caractéristiques d'ImageNet sont visiblement très adaptées à nos images.

5.3.8 Phase 2 — Fine-tuning

Les premières couches détectent des concepts très généraux (bords, couleurs) : on les garde intactes. On dégèle les **derniers blocs MBConv** (features de haut niveau) et on entraîne avec un *learning rate* très faible pour ajuster finement les filtres pré-entraînés sans les détruire. Environ 2 millions de paramètres deviennent entraînaibles.

Phase 2 — Résultats

accuracy :0,9940 | auc :0,9992 | loss :0,0232
 val_accuracy :0,9930 | val_auc :0,9997 | val_loss :0,0192

```
Trainable parameters after unfreezing: 2,038,801
Epoch 1/10
177/177 ----- 66s 206ms/step - accuracy: 0.9811 - auc: 0.9970 - loss: 0.0702 - val_accuracy: 0.9910 - val_auc: 0.9971 - val_loss: 0.0421 - learning_rate: 1.0000e-05
Epoch 2/10
177/177 ----- 37s 43ms/step - accuracy: 0.9889 - auc: 0.9980 - loss: 0.0481 - val_accuracy: 0.9910 - val_auc: 0.9977 - val_loss: 0.0351 - learning_rate: 1.0000e-05
Epoch 3/10
177/177 ----- 8s 44ms/step - accuracy: 0.9864 - auc: 0.9977 - loss: 0.0460 - val_accuracy: 0.9920 - val_auc: 0.9985 - val_loss: 0.0324 - learning_rate: 1.0000e-05
Epoch 4/10
177/177 ----- 7s 42ms/step - accuracy: 0.9910 - auc: 0.9983 - loss: 0.0361 - val_accuracy: 0.9930 - val_auc: 0.9988 - val_loss: 0.0280 - learning_rate: 1.0000e-05
Epoch 5/10
177/177 ----- 8s 43ms/step - accuracy: 0.9913 - auc: 0.9984 - loss: 0.0339 - val_accuracy: 0.9900 - val_auc: 0.9989 - val_loss: 0.0288 - learning_rate: 1.0000e-05
Epoch 6/10
177/177 ----- 8s 43ms/step - accuracy: 0.9927 - auc: 0.9986 - loss: 0.0297 - val_accuracy: 0.9920 - val_auc: 0.9992 - val_loss: 0.0255 - learning_rate: 1.0000e-05
Epoch 7/10
177/177 ----- 8s 44ms/step - accuracy: 0.9915 - auc: 0.9987 - loss: 0.0299 - val_accuracy: 0.9930 - val_auc: 0.9992 - val_loss: 0.0242 - learning_rate: 1.0000e-05
Epoch 8/10
177/177 ----- 8s 44ms/step - accuracy: 0.9924 - auc: 0.9988 - loss: 0.0277 - val_accuracy: 0.9920 - val_auc: 0.9994 - val_loss: 0.0222 - learning_rate: 1.0000e-05
Epoch 9/10
177/177 ----- 8s 44ms/step - accuracy: 0.9920 - auc: 0.9989 - loss: 0.0271 - val_accuracy: 0.9940 - val_auc: 0.9996 - val_loss: 0.0204 - learning_rate: 1.0000e-05
Epoch 10/10
177/177 ----- 8s 45ms/step - accuracy: 0.9940 - auc: 0.9992 - loss: 0.0232 - val_accuracy: 0.9930 - val_auc: 0.9997 - val_loss: 0.0192 - learning_rate: 1.0000e-05
```

Figure 5.18 — Logs d'entraînement — Phase 2 (*fine-tuning*).

Bilan : +1,9 % d'*accuracy* (97,5 % → 99,4 %) et la *val_loss* divisée par ~ 3 (de 0,0625 à 0,0192).

5.3.9 Courbes d'entraînement

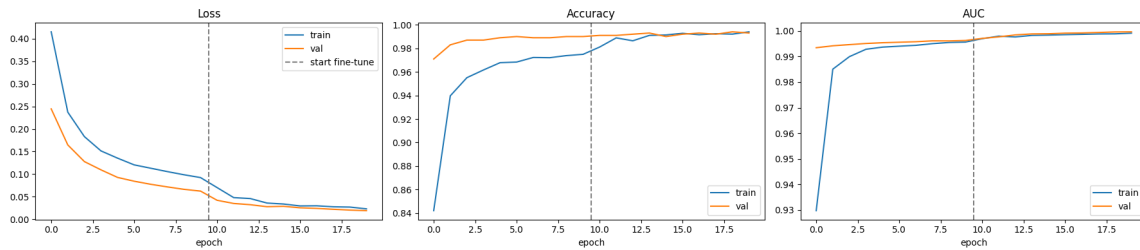


Figure 5.19 – Courbes *accuracy* / *loss* / AUC durant les phases 1 et 2.

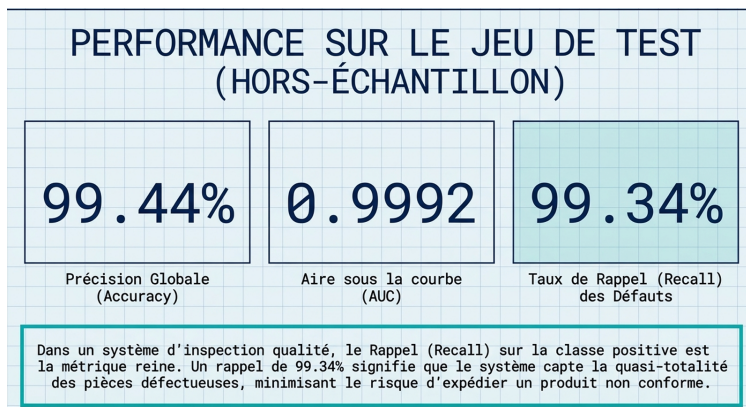
Lecture des courbes.

- L'*accuracy* de validation monte rapidement, la *loss* chute : les features ImageNet se transfèrent très bien.
- Les courbes *train* et *validation* restent proches : **pas de sur-apprentissage**.
- La transition Phase 1 $\rightarrow$ Phase 2 est visible et apporte un gain mesurable.
- L'AUC $\approx 0,99$ signifie que pour deux images tirées au hasard (une défectueuse, une OK), le modèle les sépare presque parfaitement — performance *excellente* pour le contrôle qualité industriel.

6. Évaluation finale et discussion

6.1 Performance sur l'ensemble de test

L'évaluation finale est réalisée sur les **715 images** du *test set*, jamais vues par le modèle (453 défectueuses, 262 conformes).



(a) Rapport de classification *scikit-learn*

```
Test loss      : 0.0202
Test accuracy  : 0.9944
Test AUC       : 0.9992

Classification report (positive class = def_front):
```

	precision	recall	f1-score	support
ok_front	0.9886	0.9962	0.9924	262
def_front	0.9978	0.9934	0.9956	453
accuracy			0.9944	715
macro avg	0.9932	0.9948	0.9940	715
weighted avg	0.9944	0.9944	0.9944	715

(b) Score global

Figure 6.1 – Performance détaillée sur le *test set*.

Métriques industrielles clés

Precision sur `def_front` : 99,78 %

⇒ Quand le modèle dit “défectueux”, il a raison dans 99,78 % des cas — très peu de fausses alertes (donc peu d’arrêts de production inutiles).

Recall sur `def_front` : 99,34 %

⇒ C’est **LA** métrique critique en contrôle qualité. Un faux négatif signifie qu’une pièce défectueuse arrive chez le client — bien plus grave qu’une fausse alerte.

6.2 Matrice de confusion

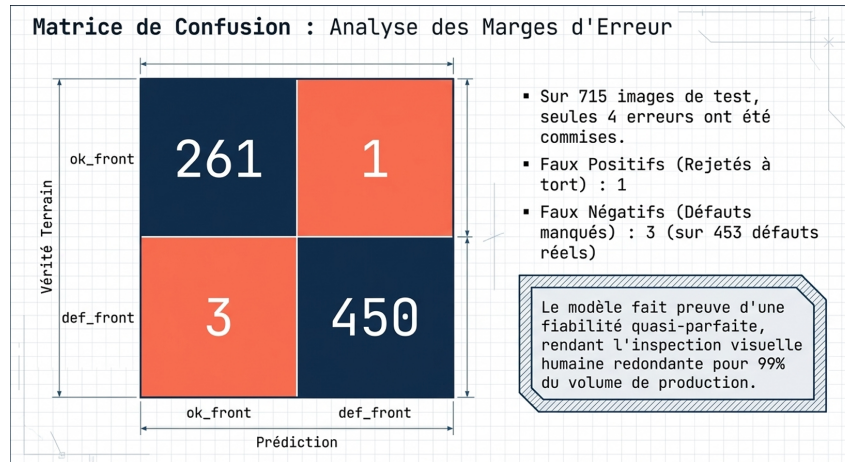


Figure 6.2 – Matrice de confusion sur le *test set*.

6.3 Courbes ROC et Precision-Recall

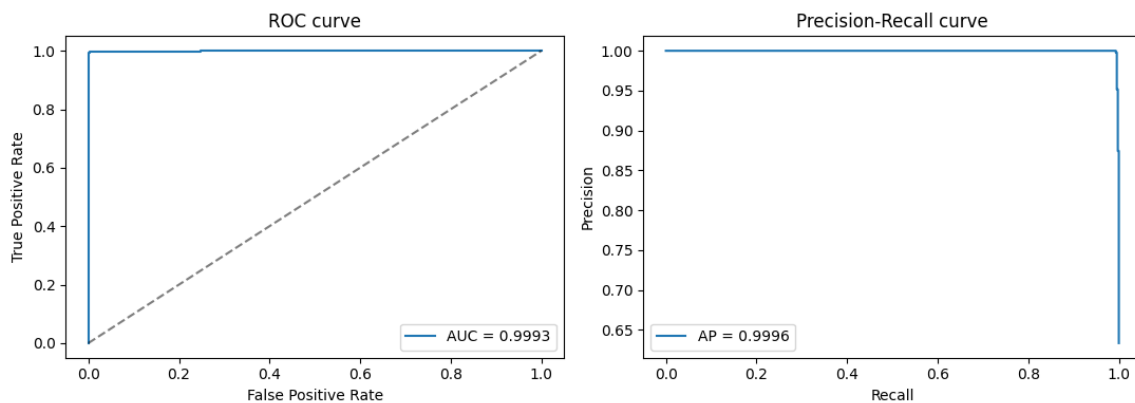


Figure 6.3 – Courbes ROC (gauche) et Precision-Recall (droite).

Lecture.

- La courbe **ROC** atteint $AUC = 0,9993$: pour tout seuil de décision, le modèle détecte presque tous les défauts (TPR élevé) tout en signalant presque zéro fausse alarme (FPR bas). La courbe colle au coin supérieur gauche — comportement *idéal*.
- La courbe **Precision-Recall** est plus informative en cas de classes déséquilibrées (453 *vs.* 262). L'*Average Precision* de 0,9996 confirme que la précision reste quasi-parfaite même pour des *recalls* très élevés.

6.4 Analyse des erreurs

Nous parcourons l'ensemble du *test set* pour identifier les erreurs et les répartir en deux catégories :

- **FN** — **Faux Négatifs** ($true = \text{def_front}$, $pred = \text{ok_front}$) : pièces défectueuses laissées passer. **Plus critique en QC.**
- **FP** — **Faux Positifs** ($true = \text{ok_front}$, $pred = \text{def_front}$) : pièces saines signalées comme défectueuses. Génèrent des fausses alertes.

Pour chaque image, on affiche la probabilité *sigmoid* p : $p \rightarrow 1$ signifie “défectueux” avec haute confiance, $p \rightarrow 0$ “OK”, et $p \approx 0,5$ une zone d'incertitude.

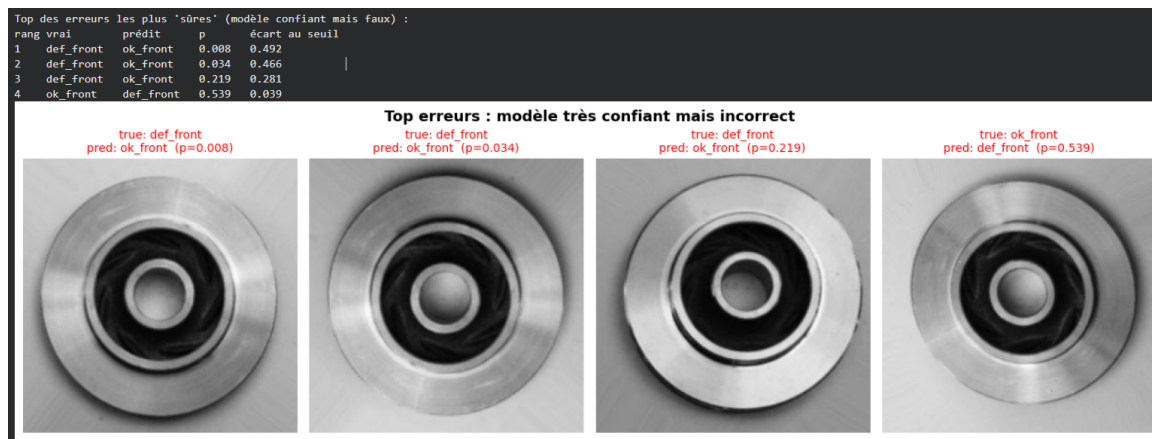


Figure 6.4 – Visualisation des images mal classées (FN à gauche, FP à droite).

6.5 Comparaison globale des trois modèles

Métrique	MLP	CNN	EfficientNetB0 (TL)
Accuracy (test)	68,02 %	$\sim 95\%^{\dagger}$	99,44 %
AUC	—	—	0,9993
Recall def_front	$\sim 68\%$	—	99,34 %
Sur-apprentissage	Faible	Présent (corrigé)	Aucun
Coût d'entraînement	Faible	Modéré	Faible (<i>frozen</i>)

Table 6.1 – Comparatif des trois approches. † après *data augmentation*.

	● MLP	● CNN augmenté	● Transfer Learning
Verdict	Sous-performant	Acceptable	Production-ready
Accuracy	68.02%	84.26%	99.44%
Recall	68.02%	—	99.34%
AUC	—	—	0.9992

Figure 6.5 — Comparaison des courbes ROC — le Transfer Learning discrimine quasi-parfaitement les deux classes, quel que soit le seuil.

7. Conclusion et perspectives

7.1 Bilan

Avec seulement $\sim 6\,600$ images d'entraînement et un GPU Colab, nous avons atteint 99,44 % d'**accuracy** et un **AUC** de 0,9993 grâce au Transfer Learning sur EFFICIENTNETB0. Sans Transfer Learning, atteindre la même performance aurait nécessité des dizaines de milliers d'images et plusieurs jours d'entraînement.

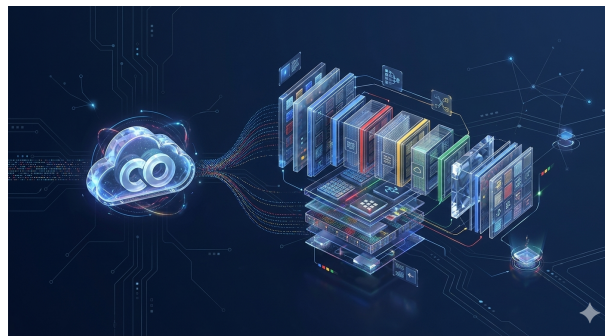


Figure 7.1 – Synthèse visuelle du résultat final.

Apports du projet

- Pipeline de prétraitement *cross-framework* (TensorFlow et PyTorch).
- Démarche progressive et pédagogique : MLP $\rightarrow$ CNN $\rightarrow$ Transfer Learning.
- Modèle final compatible avec un déploiement industriel temps réel.
- *Recall* de 99,34 % sur la classe défectueuse — adapté aux exigences QC.

7.2 Perspectives

Plusieurs pistes peuvent être explorées pour aller plus loin :

- **Multi-classes** : distinguer plusieurs types de défauts (fissure, déformation, bavure, porosité).
- **Localisation** : ajouter une détection d'objet (YOLO, Faster R-CNN) ou une segmentation pour localiser le défaut.
- **Embedded deployment** : quantification et export vers TENSORFLOW LITE pour exécution sur un Raspberry Pi ou un Edge TPU en bord de ligne.

- **Robustesse industrielle** : collecte d'images réelles de l'usine pour valider la généralisation *in situ*.
- **Active learning** : enrichissement progressif du dataset avec les cas incertains détectés en production.

Merci pour votre attention.